SENATI

Escuela de Tecnologías de la Información

GUÍA DE LABORATORIO

Guía 11

Redes Neuronales Artificiales: Estructura y Creación

Define la estructura y crea una red neuronal artificial

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Curso	Seminario de Complementación Práctica I (PIAD-427)
Semana	11
Tarea de aprendizaje	Define la estructura y crea una red neuronal artificial
Herramientas	Python 3 · Visual Studio Code · TensorFlow · Keras · NumPy
Instructor	Rembrandt Ubalde Enríquez
Estudiante	
Fecha	

Contenido

Objetivos de la guía 3
Materiales 3
Parte 1. Preparación del entorno en Visual Studio Code 4
1.1 Crear la carpeta del proyecto 4
1.2 Crear y activar el entorno virtual 4
1.3 Instalar las librerías 4
1.4 Seleccionar el intérprete y ejecutar scripts 5
Parte 2. Conceptos clave 6
2.1 ¿Qué es una red neuronal artificial y por qué es importante? 6
2.2 Estructura de una red neuronal 6
2.3 Funciones de activación 7
2.4 Capas ocultas (conocimiento complementario) 7
2.5 Tipos de redes neuronales artificiales 7
2.6 Pasos para crear una red con TensorFlow y Keras 7
Parte 3. Prácticas en Visual Studio Code 9
Práctica 01. Una neurona artificial con NumPy 9
Práctica 02. Funciones de activación 10
Práctica 03. Estructura de una red: neuronas, capas y conexiones 11
Práctica 04. Mi primera red neuronal con TensorFlow y Keras 11
Práctica 05. ¿Qué aportan las capas ocultas? (complementario) 13
Práctica 06. Los cuatro tipos de redes en Keras 14
Práctica 07. Simulador TensorFlow Playground (complementario) 16
Parte 4. Errores frecuentes 18
Parte 5. Ejercicios propuestos 18
Preguntas de reflexión 18
Lista de cotejo 19

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Objetivos de la guía

Al terminar esta guía serás capaz de:

- **Definir** qué es una red neuronal artificial y explicar su importancia en la inteligencia artificial.
- **Describir** la estructura de una red neuronal: neuronas, capas, conexiones y funciones de activación.
- **Identificar** los tipos de redes neuronales: Feedforward, Recurrentes, Convolucionales y Generativas Adversarias.
- **Crear** una red neuronal con TensorFlow y Keras en Visual Studio Code.

Operación	Dónde se trabaja en esta guía
Define la red neuronal artificial y su importancia en la IA	Parte 2 (sección 2.1) y Práctica 01
Describe la estructura de una red neuronal artificial	Parte 2 (secciones 2.2 y 2.3), Prácticas 02 y 03
Identifica los tipos de redes neuronales artificiales	Parte 2 (sección 2.5) y Práctica 06
Crea una red neuronal con TensorFlow y Keras	Parte 2 (sección 2.6) y Práctica 04
Capas ocultas y simuladores (complementario)	Parte 2 (sección 2.4), Prácticas 05 y 07

Materiales

- Computadora con Windows 10/11 de **64 bits**, Python 3.10 a 3.12 y Visual Studio Code con las extensiones **Python** y **Pylance**.
- Conexión a internet para instalar TensorFlow (unos 400 MB, solo la primera vez) y para usar el simulador TensorFlow Playground.

Parte 1. Preparación del entorno en Visual Studio Code

Si ya trabajaste las guías anteriores, tienes Python y VS Code listos. Igual sigue estos pasos para crear la **carpeta de esta guía** y su **entorno virtual**.

1.1 Crear la carpeta del proyecto

1. Crea en Documentos una carpeta llamada **IA_Guia11_Redes_Neuronales**.
2. Abre Visual Studio Code y entra a **Archivo > Abrir carpeta…** (Ctrl + K, Ctrl + O). Selecciona la carpeta.
3. Si VS Code pregunta **«¿Confía en los autores de los archivos?»**, elige **Sí, confío en los autores**.
4. Abre la terminal integrada con **Terminal > Nuevo terminal** (Ctrl + ñ).

1.2 Crear y activar el entorno virtual

Terminal de VS Code

```
python -m venv venv
.\venv\Scripts\activate
```

Cuando el entorno está activo, la terminal muestra **(venv)** al inicio de la línea.

> ⚠ **Atención:** Si PowerShell muestra **«la ejecución de scripts está deshabilitada en este sistema»**, ejecuta una sola vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, responde **S** y vuelve a activar el entorno.

1.3 Instalar las librerías

Terminal de VS Code

```
pip install tensorflow numpy matplotlib scikit-learn
```

> ⚠ **Atención:** TensorFlow solo se instala en Python de **64 bits** y con versiones compatibles. Si `pip` responde **«No matching distribution found for tensorflow»**, revisa tu versión con `python --version`; las versiones 3.10, 3.11 y 3.12 son las más seguras. Puedes consultar la tabla oficial en tensorflow.org/install.

Verifica la instalación:

Terminal de VS Code

```
python -c "import tensorflow as tf; print(tf.__version__)"
```

▶ Resultado esperado en la terminal

```
2.x.x   (o una versión más reciente)
```

1.4 Seleccionar el intérprete y ejecutar scripts

1. Pulsa **Ctrl + Shift + P**, escribe **Python: Select Interpreter** y elige el que dice **('venv': venv)**.
2. Crea cada archivo con el ícono **Nuevo archivo** del Explorador, con el nombre exacto que indica la guía (por ejemplo `01_neurona.py`).
3. Copia el código y guarda con **Ctrl + S**.
4. Ejecuta con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo `python nombre_del_archivo.py` en la terminal.
5. Compara lo que aparece en la terminal con el recuadro verde **Resultado esperado**.

> 💡 **Consejo:** Cuando un script abre una ventana (`plt.show()` o `cv2.imshow()`), el programa queda en pausa hasta que **cierres la ventana** o **presiones una tecla** sobre ella. Las imágenes también quedan guardadas como PNG en tu carpeta.

> 💡 **Consejo:** Para depurar paso a paso: haz clic a la izquierda del número de línea para poner un **punto de interrupción** (punto rojo) y pulsa **F5 > Python File**. En el panel **Variables** verás los valores de las matrices mientras el programa avanza.

> 💡 **Consejo:** La primera vez que importas TensorFlow puede tardar varios segundos y mostrar mensajes como **«oneDNN custom operations are on»** o avisos sobre la GPU. Son **informativos**, no errores. Si quieres ocultarlos, escribe en la terminal `$env:TF_CPP_MIN_LOG_LEVEL="2"` antes de ejecutar.

Parte 2. Conceptos clave

2.1 ¿Qué es una red neuronal artificial y por qué es importante?

Una **red neuronal artificial (RNA)** es un modelo de aprendizaje automático inspirado en el cerebro humano. Está formada por muchas unidades simples llamadas **neuronas**, organizadas en **capas** y unidas por **conexiones con pesos**. Durante el entrenamiento, la red ajusta esos pesos para que su salida se parezca cada vez más a la respuesta correcta.

Neurona biológica	Neurona artificial
Dendritas: reciben señales de otras neuronas.	**Entradas** x1, x2, …, xn.
Sinapsis: fortalecen o debilitan la señal.	**Pesos** w1, w2, …, wn.
Cuerpo celular: suma las señales.	**Suma ponderada** z = Σ xi·wi + b.
Se «dispara» si la señal supera un umbral.	**Función de activación** f(z).
Axón: envía la señal a otras neuronas.	**Salida** que pasa a la siguiente capa.

Importancia en la IA: las redes neuronales son la base del **Deep Learning** (aprendizaje profundo). Gracias a ellas existen el reconocimiento facial, los asistentes de voz, la traducción automática, los autos autónomos, el diagnóstico por imágenes médicas y los modelos de lenguaje como los chatbots. Su ventaja es que **aprenden solas las características** importantes de los datos, sin que una persona tenga que programarlas.

2.2 Estructura de una red neuronal

Elemento	Descripción
Neurona	Unidad que recibe entradas, calcula `z = Σ x·w + b` y aplica una función de activación.
Capa de entrada	Recibe los datos. Tiene **una neurona por cada característica** (por ejemplo, 4 medidas de una flor = 4 neuronas).
Capas ocultas	Están entre la entrada y la salida. Aquí la red **aprende patrones**. Una red con varias capas ocultas es una red **profunda**.
Capa de salida	Entrega el resultado: 1 neurona para regresión o clasificación binaria; **una por clase** en clasificación múltiple.
Conexiones (pesos)	Cada neurona de una capa se une con las de la siguiente. Cada conexión tiene un **peso** que indica su importancia.
Sesgo (bias)	Valor que se suma para desplazar la activación; permite que la neurona se active aunque las entradas sean 0.
Parámetros	Total de pesos + sesgos. En una capa densa: **entradas × neuronas + neuronas**.

Ejemplo: red 4 – 10 – 8 – 3 (Práctica 04)

```
Capa oculta 1:  4 entradas × 10 neuronas + 10 sesgos =  50 parámetros
Capa oculta 2: 10 entradas ×  8 neuronas +  8 sesgos =  88 parámetros
Capa de salida: 8 entradas ×  3 neuronas +  3 sesgos =  27 parámetros
                                          TOTAL = 165 parámetros
```

2.3 Funciones de activación

La función de activación decide **cuánto se activa** una neurona. Sin ella, la red solo podría aprender relaciones lineales (rectas), por muchas capas que tenga.

Función	Fórmula / rango	Dónde se usa
Sigmoide	1 / (1 + e^-z) · de 0 a 1	Salida de clasificación **binaria** (probabilidad).
Tanh	de -1 a 1	Capas ocultas y redes recurrentes.
ReLU	max(0, z) · de 0 a ∞	La más usada en **capas ocultas**: rápida y simple.
Softmax	Convierte varios valores en probabilidades que suman 1	Salida de clasificación **múltiple** (3 o más clases).

2.4 Capas ocultas (conocimiento complementario)

- **Sin capas ocultas**, la red solo traza fronteras rectas, como una regresión logística.
- **Con una capa oculta** ya puede aprender curvas; con **varias**, combina patrones simples (bordes, trazos) en patrones complejos (formas, objetos).
- **Más capas y neuronas no siempre es mejor**: aumentan los parámetros, el tiempo de entrenamiento y el riesgo de **sobreajuste** (memorizar en vez de aprender).
- Regla práctica: empieza con 1 o 2 capas ocultas con un número de neuronas entre el de la entrada y el de la salida, y ajusta según los resultados.

2.5 Tipos de redes neuronales artificiales

Tipo	Cómo funciona	Para qué se usa	Capa en Keras
Feedforward (FNN)	La información va en un solo sentido: entrada → ocultas → salida.	Datos en tablas: aprobar un crédito, predecir una nota.	`Dense`
Recurrente (RNN)	Tiene «memoria»: la salida de un paso se usa en el siguiente.	Secuencias: texto, voz, series de tiempo, predicción de ventas.	`SimpleRNN`, `LSTM`, `GRU`
Convolucional (CNN)	Aplica **filtros** que recorren la imagen y detectan bordes, texturas y formas.	Imágenes y video: reconocimiento facial, detección de objetos.	`Conv2D`, `MaxPooling2D`
Generativa Adversaria (GAN)	Dos redes compiten: el **generador** crea datos falsos y el **discriminador** intenta detectarlos.	Crear imágenes realistas, aumentar datos, mejorar fotos.	Dos modelos `Sequential`

2.6 Pasos para crear una red con TensorFlow y Keras

Paso	Qué se hace	Código clave
1. Preparar datos	Cargar, escalar y dividir en entrenamiento y prueba.	`StandardScaler`, `train_test_split`

Paso	Qué se hace	Código clave
2. Definir la estructura	Capas, neuronas y activaciones.	`keras.Sequential([...])`
3. Compilar	Elegir optimizador, función de pérdida y métrica.	`modelo.compile()`
4. Entrenar	La red ajusta sus pesos durante varias **épocas**.	`modelo.fit()`
5. Evaluar	Medir la exactitud con datos que no vio.	`modelo.evaluate()`
6. Predecir y guardar	Usar el modelo con datos nuevos y guardarlo.	`modelo.predict()`, `modelo.save()`

Término	Significado
Época (epoch)	Una pasada completa por todos los datos de entrenamiento.
Lote (batch_size)	Cantidad de ejemplos que la red procesa antes de actualizar sus pesos.
Función de pérdida (loss)	Mide qué tan equivocada está la red. Debe **bajar** durante el entrenamiento.
Optimizador (adam)	Algoritmo que decide cómo corregir los pesos usando el error (retropropagación).

Parte 3. Prácticas en Visual Studio Code

Práctica 01. Una neurona artificial con NumPy

Programarás una sola neurona que decide si un estudiante aprueba según su asistencia, sus horas de juego y sus tareas entregadas (valores entre 0 y 1).

Archivo: 01_neurona.py

```
# 01_neurona.py - Una neurona artificial programada con NumPy
import numpy as np

# Entradas: características de un estudiante (valores entre 0 y 1)
x = np.array([0.8, 0.3, 0.5])        # x1 = asistencia, x2 = horas de juego, x3 =
tareas entregadas
# Pesos: importancia de cada entrada (los "aprende" la red)
w = np.array([0.6, -0.4, 0.9])
b = -0.2                              # sesgo (bias)

# 1. Suma ponderada: z = x1·w1 + x2·w2 + x3·w3 + b
z = np.dot(x, w) + b
print("Entradas x :", x)
print("Pesos   w :", w)
print("Sesgo   b :", b)
print(f"Suma ponderada z = {z:.3f}")

# 2. Función de activación
sigmoide = 1 / (1 + np.exp(-z))
relu = max(0, z)
print(f"Salida con sigmoide : {sigmoide:.4f}")
print(f"Salida con ReLU     : {relu:.4f}")

# 3. Decisión final
print("Predicción:", "APRUEBA" if sigmoide >= 0.5 else "DESAPRUEBA")
```

▶ Resultado esperado en la terminal

```
Entradas x : [0.8 0.3 0.5]
Pesos   w : [ 0.6 -0.4  0.9]
Sesgo   b : -0.2
Suma ponderada z = 0.610
Salida con sigmoide : 0.6479
Salida con ReLU     : 0.6100
Predicción: APRUEBA
```

Análisis: z = 0.8·0.6 + 0.3·(-0.4) + 0.5·0.9 - 0.2 = **0.61**. El peso negativo de las horas de juego **resta**: más juego reduce la probabilidad de aprobar. La sigmoide convierte 0.61 en una probabilidad de **64.8 %**.

Práctica 02. Funciones de activación

Archivo: 02_activaciones.py

```
# 02_activaciones.py - Funciones de activación más usadas
import numpy as np
import matplotlib.pyplot as plt

def sigmoide(z): return 1 / (1 + np.exp(-z))
def tanh(z):     return np.tanh(z)
def relu(z):     return np.maximum(0, z)
def softmax(z):
    e = np.exp(z - np.max(z))
    return e / e.sum()

print("   z    sigmoide   tanh    ReLU")
for z in [-3, -1, 0, 1, 3]:
    print(f"{z:>4} {sigmoide(z):>9.3f} {tanh(z):>7.3f} {relu(z):>6.1f}")

puntajes = np.array([2.0, 1.0, 0.1])          # salida "cruda" de 3 neuronas
prob = softmax(puntajes)
print("\nSoftmax de", puntajes, "->", prob.round(3), "| suma =", prob.sum().round(2))

z = np.linspace(-5, 5, 200)
plt.figure(figsize=(9, 3))
for i, (nombre, f) in enumerate([("Sigmoide", sigmoide), ("Tanh", tanh), ("ReLU",
relu)]):
    plt.subplot(1, 3, i + 1)
    plt.plot(z, f(z), linewidth=2)
    plt.title(nombre); plt.grid(alpha=0.3); plt.axhline(0, color="gray", lw=0.8)
plt.tight_layout()
plt.savefig("g11_activaciones.png", dpi=110, bbox_inches="tight")
plt.show()
```

▶ Resultado esperado en la terminal

```
   z    sigmoide   tanh    ReLU
  -3     0.047  -0.995    0.0
  -1     0.269  -0.762    0.0
   0     0.500   0.000    0.0
   1     0.731   0.762    1.0
   3     0.953   0.995    3.0

Softmax de [2.  1.  0.1] -> [0.659 0.242 0.099] | suma = 1.0
```

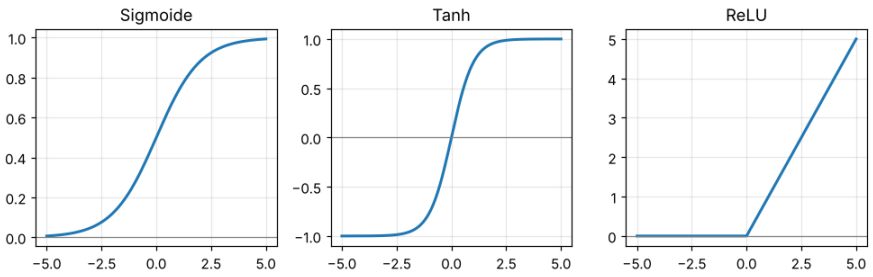


Figura 1. Sigmoide (0 a 1), Tanh (-1 a 1) y ReLU (0 para negativos).

Observa que ReLU deja pasar los valores positivos tal cual y convierte los negativos en 0. Softmax transformó los puntajes 2.0, 1.0 y 0.1 en probabilidades que **suman 1**.

Práctica 03. Estructura de una red: neuronas, capas y conexiones

Construirás a mano una red de **3 neuronas de entrada, 4 ocultas y 2 de salida**, y harás la **propagación hacia adelante** (forward): los datos pasan capa por capa hasta la salida.

Archivo: 03_red_numpy.py

```
# 03_red_numpy.py - Estructura de una red 3-4-2: neuronas, capas y conexiones
import numpy as np

np.random.seed(1)
# Capa de entrada: 3 neuronas | Capa oculta: 4 neuronas | Capa de salida: 2 neuronas
W1 = np.random.randn(3, 4).round(2)   # conexiones entrada -> oculta (3 x 4 = 12 pesos)
b1 = np.zeros(4)                      # 4 sesgos
W2 = np.random.randn(4, 2).round(2)   # conexiones oculta -> salida (4 x 2 = 8 pesos)
b2 = np.zeros(2)                      # 2 sesgos

def relu(z): return np.maximum(0, z)
def softmax(z):
    e = np.exp(z - z.max())
    return e / e.sum()

x = np.array([0.5, 0.1, 0.9])         # un ejemplo de entrada
h = relu(x @ W1 + b1)                 # propagación hacia adelante: capa oculta
y = softmax(h @ W2 + b2)              # capa de salida

print("Pesos W1 (entrada -> oculta):\n", W1)
print("Salida de la capa oculta h:", h.round(3))
print("Pesos W2 (oculta -> salida):\n", W2)
print("Probabilidades de salida y:", y.round(3))
print("Clase predicha:", y.argmax())
parametros = W1.size + b1.size + W2.size + b2.size
print(f"Total de parámetros: {W1.size} + {b1.size} + {W2.size} + {b2.size} =
{parametros}")
```

▶ Resultado esperado en la terminal

```
Pesos W1 (entrada -> oculta):
 [[ 1.62 -0.61 -0.53 -1.07]
 [ 0.87 -2.3   1.74 -0.76]
 [ 0.32 -0.25  1.46 -2.06]]
Salida de la capa oculta h: [1.185 0.    1.223 0.   ]
Pesos W2 (oculta -> salida):
 [[-0.32 -0.38]
 [ 1.13 -1.1 ]
 [-0.17 -0.88]
 [ 0.04  0.58]]
Probabilidades de salida y: [0.719 0.281]
Clase predicha: 0
Total de parámetros: 12 + 4 + 8 + 2 = 26
```

Análisis: la matriz W1 tiene **3 × 4 = 12** conexiones y W2 tiene **4 × 2 = 8**. Dos neuronas ocultas quedaron en 0 porque su suma fue negativa y ReLU las «apagó». Los pesos aún son **aleatorios**: la red no está entrenada. Entrenar es justamente encontrar los pesos correctos, y eso lo hará Keras por nosotros.

Práctica 04. Mi primera red neuronal con TensorFlow y Keras

Crearás la red **4 – 10 – 8 – 3** de la sección 2.2 para clasificar las 3 especies de flores Iris, siguiendo los seis pasos de la sección 2.6.

Archivo: 04_keras_iris.py

```
# 04_keras_iris.py - Mi primera red neuronal con TensorFlow y Keras
import numpy as np
from tensorflow import keras
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

keras.utils.set_random_seed(42)           # resultados repetibles

# 1. DATOS: 150 flores, 4 medidas, 3 especies
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
escalador = StandardScaler()
X_tr = escalador.fit_transform(X_tr)
X_te = escalador.transform(X_te)

# 2. ESTRUCTURA: entrada(4) -> oculta(10) -> oculta(8) -> salida(3)
modelo = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(10, activation="relu", name="oculta_1"),
    keras.layers.Dense(8, activation="relu", name="oculta_2"),
    keras.layers.Dense(3, activation="softmax", name="salida"),
])
for capa in modelo.layers:
    print(f"{capa.name:<9} neuronas={capa.units:<3}
activación={capa.activation.__name__:<8} parámetros={capa.count_params()}")
print("Total de parámetros:", modelo.count_params())

# 3. COMPILAR: optimizador, función de pérdida y métrica
modelo.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

# 4. ENTRENAR
hist = modelo.fit(X_tr, y_tr, epochs=100, batch_size=8, verbose=0)
for ep in [1, 25, 50, 100]:
    print(f"Época {ep:>3}: pérdida = {hist.history['loss'][ep-1]:.4f} | exactitud =
{hist.history['accuracy'][ep-1]:.4f}")

# 5. EVALUAR con datos de prueba
perdida, exactitud = modelo.evaluate(X_te, y_te, verbose=0)
print(f"Exactitud en prueba: {exactitud:.2%}")

# 6. PREDECIR una flor nueva
nueva = escalador.transform([[6.3, 2.9, 5.6, 1.8]])
prob = modelo.predict(nueva, verbose=0)[0]
especies = ["setosa", "versicolor", "virginica"]
print("Probabilidades:", {e: round(float(p), 3) for e, p in zip(especies, prob)})
print("Especie predicha:", especies[np.argmax(prob)])

modelo.save("modelo_iris.keras")
print("Modelo guardado en modelo_iris.keras")
```

▶ **Resultado aproximado (los decimales pueden variar en tu PC)**

```
oculta_1  neuronas=10  activación=relu     parámetros=50
oculta_2  neuronas=8   activación=relu     parámetros=88
salida    neuronas=3   activación=softmax  parámetros=27
Total de parámetros: 165
Época   1: pérdida = 1.2104 | exactitud = 0.3417
Época  25: pérdida = 0.3015 | exactitud = 0.8833
Época  50: pérdida = 0.1102 | exactitud = 0.9583
Época 100: pérdida = 0.0587 | exactitud = 0.9750
Exactitud en prueba: 96.67%
Probabilidades: {'setosa': 0.001, 'versicolor': 0.019, 'virginica': 0.98}
Especie predicha: virginica
Modelo guardado en modelo_iris.keras
```

Análisis: las cantidades de parámetros coinciden con el cálculo de la sección 2.2. Durante el entrenamiento la **pérdida baja** (de 1.2 a 0.06) y la **exactitud sube**: la red está aprendiendo. En prueba acierta alrededor del **97 %** (29 de 30 flores). Para la flor nueva asigna 98 % de probabilidad a **virginica**.

⚠ **Atención:** Aunque fijamos la semilla con `set_random_seed(42)`, los valores de pérdida y exactitud pueden cambiar en algunos decimales según la versión de TensorFlow y el procesador. Lo importante es que la pérdida **baje** y la exactitud en prueba quede **por encima del 90 %**.

💡 **Consejo:** Agrega `modelo.summary()` después de crear el modelo para ver la tabla resumen que trae Keras, con la forma de salida y los parámetros de cada capa.

Práctica 05. ¿Qué aportan las capas ocultas? (complementario)

El dataset **«dos lunas»** tiene dos grupos de puntos con forma de media luna que **no se pueden separar con una recta**. Compararás tres redes: sin capas ocultas, con una y con dos.

Archivo: 05_capas_ocultas.py

```
# 05_capas_ocultas.py - ¿Qué aportan las capas ocultas? (dataset de dos lunas)
from tensorflow import keras
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=500, noise=0.25, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)

def crear_red(ocultas):
    capas = [keras.Input(shape=(2,))]
    for n in ocultas:
        capas.append(keras.layers.Dense(n, activation="relu"))
    capas.append(keras.layers.Dense(1, activation="sigmoid"))     # salida binaria
    return keras.Sequential(capas)

configuraciones = {"Sin capas ocultas": [], "1 capa oculta (8)": [8], "2 capas ocultas
(16, 8)": [16, 8]}
print(f"{'Arquitectura':<25}{'Parámetros':>11}{'Exactitud':>11}")
for nombre, ocultas in configuraciones.items():
    keras.utils.set_random_seed(0)
    red = crear_red(ocultas)
    red.compile(optimizer=keras.optimizers.Adam(0.01), loss="binary_crossentropy",
metrics=["accuracy"])
    red.fit(X_tr, y_tr, epochs=150, batch_size=32, verbose=0)
    _, acc = red.evaluate(X_te, y_te, verbose=0)
    print(f"{nombre:<25}{red.count_params():>11}{acc:>11.2%}")
```

▶ Resultado aproximado (los decimales pueden variar en tu PC)

```
Arquitectura              Parámetros  Exactitud
Sin capas ocultas                  3      ≈ 83%
1 capa oculta (8)                 33      ≈ 90%
2 capas ocultas (16, 8)          193      ≈ 92%
```

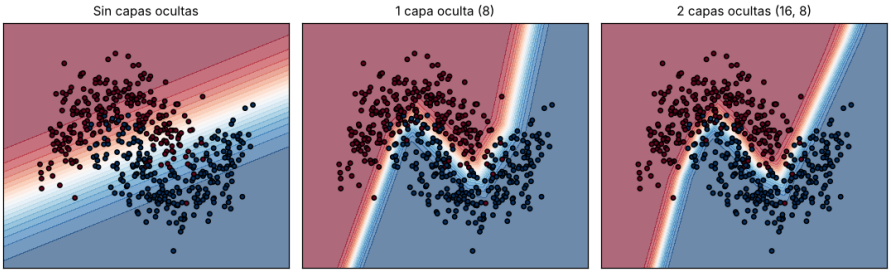


Figura 2. Frontera de decisión de cada red (figura de referencia; la tuya será muy parecida).

Conclusión: sin capas ocultas la red solo dibuja una **recta** y se equivoca en las puntas de las lunas (≈ 83 %). Con capas ocultas la frontera se **curva** y la exactitud sube a más del 90 %.

Práctica 06. Los cuatro tipos de redes en Keras

Construirás la estructura (sin entrenar) de cada tipo de red de la sección 2.5 para comparar sus capas y su tamaño.

Archivo: 06_tipos_redes.py

```
# 06_tipos_redes.py - Estructura de los 4 tipos de redes neuronales en Keras
from tensorflow import keras
from tensorflow.keras import layers

def describir(nombre, modelo):
    print(f"--- {nombre} ---")
    for capa in modelo.layers:
        print(f"  {capa.__class__.__name__:<13} salida={str(capa.output.shape):<18}
parámetros={capa.count_params()}")
    print(f"  TOTAL: {modelo.count_params()} parámetros\n")

# 1. FEEDFORWARD (FNN): datos tabulares, la información va en un solo sentido
fnn = keras.Sequential([keras.Input(shape=(20,)),
                        layers.Dense(16, activation="relu"),
                        layers.Dense(1, activation="sigmoid")])
describir("Feedforward (FNN)", fnn)

# 2. RECURRENTE (RNN): secuencias (texto, series de tiempo); recuerda lo anterior
rnn = keras.Sequential([keras.Input(shape=(10, 1)),          # 10 pasos de tiempo, 1
valor
                        layers.SimpleRNN(16),
                        layers.Dense(1)])
describir("Recurrente (RNN)", rnn)

# 3. CONVOLUCIONAL (CNN): imágenes; los filtros detectan bordes y formas
cnn = keras.Sequential([keras.Input(shape=(28, 28, 1)),      # imagen 28x28 en grises
                        layers.Conv2D(8, (3, 3), activation="relu"),
                        layers.MaxPooling2D((2, 2)),
                        layers.Flatten(),
                        layers.Dense(10, activation="softmax")])
describir("Convolucional (CNN)", cnn)

# 4. GAN: dos redes que compiten (generador vs. discriminador)
generador = keras.Sequential([keras.Input(shape=(16,)),       # ruido aleatorio
                              layers.Dense(32, activation="relu"),
                              layers.Dense(2)])               # dato "falso" de 2
valores
discriminador = keras.Sequential([keras.Input(shape=(2,)),
                                  layers.Dense(32, activation="relu"),
                                  layers.Dense(1, activation="sigmoid")])  # ¿real o
falso?
describir("GAN - Generador", generador)
describir("GAN - Discriminador", discriminador)
```

▶ **Resultado esperado en la terminal**

```
--- Feedforward (FNN) ---
  Dense         salida=(None, 16)         parámetros=336
  Dense         salida=(None, 1)          parámetros=17
  TOTAL: 353 parámetros

--- Recurrente (RNN) ---
  SimpleRNN     salida=(None, 16)         parámetros=288
  Dense         salida=(None, 1)          parámetros=17
  TOTAL: 305 parámetros

--- Convolucional (CNN) ---
  Conv2D        salida=(None, 26, 26, 8)  parámetros=80
  MaxPooling2D  salida=(None, 13, 13, 8)  parámetros=0
  Flatten       salida=(None, 1352)       parámetros=0
  Dense         salida=(None, 10)         parámetros=13530
  TOTAL: 13610 parámetros

--- GAN - Generador ---
  Dense         salida=(None, 32)         parámetros=544
  Dense         salida=(None, 2)          parámetros=66
  TOTAL: 610 parámetros

--- GAN - Discriminador ---
  Dense         salida=(None, 32)         parámetros=96
  Dense         salida=(None, 1)          parámetros=33
  TOTAL: 129 parámetros
```

Análisis: la CNN reduce la imagen de 28 × 28 a 26 × 26 con 8 filtros (cada filtro tiene solo 3 × 3 + 1 = 10 parámetros) y el `MaxPooling2D` la reduce a la mitad **sin parámetros**. La RNN recibe una **secuencia** de 10 pasos. La GAN está formada por **dos redes**: el generador recibe ruido y produce un dato; el discriminador recibe un dato y responde si es real o falso.

Práctica 07. Simulador TensorFlow Playground (complementario)

TensorFlow Playground es un simulador gratuito en el navegador que permite ver cómo aprende una red neuronal sin escribir código.

1. Abre el navegador y entra a **playground.tensorflow.org**.
2. A la izquierda, en **DATA**, elige el conjunto de datos **círculo** (el primero).
3. Quita todas las capas ocultas con el botón **–** de **HIDDEN LAYERS** y presiona **▶ (Play)**. Observa que la red no logra separar los colores: solo traza una recta.
4. Presiona **Reset**, agrega **1 capa oculta con 4 neuronas** y vuelve a ejecutar. Anota el **Test loss** después de 300 épocas.
5. Cambia la **Activation** entre ReLU, Tanh y Sigmoid y compara la velocidad con que baja la pérdida.
6. Elige el conjunto **espiral** (el último) e intenta lograr un Test loss menor a 0.1 combinando capas, neuronas y características de entrada.

Elemento del simulador	Concepto de la guía
FEATURES (X1, X2, …)	Capa de **entrada**.
HIDDEN LAYERS	**Capas ocultas** y número de **neuronas**.

Elemento del simulador	Concepto de la guía
Líneas entre neuronas (azules/naranjas)	**Conexiones**: el grosor es el tamaño del **peso**; azul = positivo, naranja = negativo.
OUTPUT y Test loss	Capa de **salida** y **función de pérdida**.
Epoch, Learning rate, Activation	**Épocas**, tasa de aprendizaje y **función de activación**.

> 💡 **Consejo:** Otros simuladores útiles: **Neural Network Playground** de Google (el mismo), **CNN Explainer** (poloclub.github.io/cnn-explainer) para ver cómo una CNN procesa una imagen, y **Teachable Machine** (teachablemachine.withgoogle.com) para entrenar un clasificador de imágenes con tu webcam.

Parte 4. Errores frecuentes

Mensaje o síntoma	Causa	Solución
`No matching distribution found for tensorflow`	Versión de Python no compatible o Python de 32 bits.	Instala Python 3.11 o 3.12 de 64 bits y crea de nuevo el venv.
`ModuleNotFoundError: No module named 'tensorflow'`	Se instaló en otro entorno o el intérprete no es el del venv.	Activa el venv, selecciona el intérprete **venv** y repite `pip install tensorflow`.
`DLL load failed while importing _pywrap_tensorflow_internal`	Faltan las librerías de Visual C++ en Windows.	Instala **Microsoft Visual C++ Redistributable** (x64) y reinicia VS Code.
Mensajes «oneDNN…» o «Could not find cuda drivers»	Avisos informativos; la PC no tiene GPU NVIDIA.	No es un error: TensorFlow usa el procesador (CPU).
`ValueError: Input 0 of layer … expected shape=(None, 4)`	Los datos no tienen la forma que indica `keras.Input`.	Revisa `X.shape` y que el número de columnas coincida con `shape=(4,)`.
La exactitud se queda baja y la pérdida no baja	Datos sin escalar, pocas épocas o tasa de aprendizaje inadecuada.	Aplica `StandardScaler`, aumenta `epochs` o prueba otro optimizador.

Parte 5. Ejercicios propuestos

1. Programa con NumPy un **perceptrón** (una neurona con función escalón) que aprenda la compuerta lógica **AND** usando la regla de aprendizaje del perceptrón. Muestra los pesos finales y la salida para las 4 combinaciones.
2. Calcula **a mano** el número de parámetros de una red **4 – 8 – 6 – 3** y verifica tu respuesta creando el modelo en Keras con `count_params()`.
3. En la Práctica 04 cambia la activación de las capas ocultas de `relu` a `tanh` y luego a `sigmoid`. Compara la pérdida final y la exactitud en prueba. ¿Cuál aprende más rápido?
4. Modifica la Práctica 03 para agregar una **segunda capa oculta de 3 neuronas** con activación `tanh`. ¿Cuántos parámetros tiene ahora la red? ¿Qué clase predice?
5. En **TensorFlow Playground**, con el conjunto de datos **XOR** (el segundo), encuentra la red más pequeña (menos neuronas) que logre un **Test loss menor a 0.05**. Registra la configuración y explica el resultado.

Preguntas de reflexión

1. ¿Por qué una red neuronal necesita funciones de activación no lineales?
2. ¿Qué diferencia hay entre los **pesos** y los **hiperparámetros** (épocas, neuronas, capas)?
3. ¿Qué tipo de red usarías para: a) predecir la demanda eléctrica de la próxima hora, b) detectar placas de autos en una foto, c) aprobar un préstamo? Justifica.
4. ¿Por qué agregar muchas capas ocultas puede empeorar el resultado en datos nuevos?

Lista de cotejo

- ☐ Instalé TensorFlow en el entorno virtual y verifiqué su versión.
- ☐ Explico qué es una red neuronal y su importancia en la IA.
- ☐ Programé una neurona y calculé su suma ponderada y su salida (Práctica 01).
- ☐ Diferencio sigmoide, tanh, ReLU y softmax (Práctica 02).
- ☐ Identifico neuronas, capas, conexiones y cuento los parámetros de una red (Prácticas 03 y 04).
- ☐ Creé, entrené, evalué y guardé una red con TensorFlow y Keras (Práctica 04).
- ☐ Comprobé el efecto de las capas ocultas (Práctica 05) y usé TensorFlow Playground (Práctica 07).
- ☐ Identifico los cuatro tipos de redes y sus capas en Keras (Práctica 06).
- ☐ Resolví al menos tres ejercicios propuestos.